

Author
Mohamad Khabour
Bachelor of Science
k12005283

Submission
Institut für Symbolic
Artificial Intelligence

Supervisor
Univ.-Prof.ⁱⁿ Dr.ⁱⁿ
Martina Seidl

May 2026

Systematic Testing of QBF Solvers via Truth- Value-Preserving Transformations



Bachelor Thesis
to obtain the academic degree of
Bachelor of Science (BSc)
in the Bachelor's Program
Computer Science

STATUTORY DECLARATION

I hereby declare that the thesis submitted is my own unaided work, that I have not used other than the sources indicated, and that all direct and indirect sources are acknowledged as references.

I acknowledge the use of Large Language Models (LLMs) for the purposes of stylistic enhancement. I assume full responsibility for the content of submitted work, including checking for plagiarism and the veracity of all content.

Mohamad Khabour

Linz, May 2026

ABSTRACT

Quantified Boolean Formulas (QBF) extend propositional logic with existential and universal quantifiers, making the satisfiability problem PSPACE-complete. Specialized solvers such as DepQBF and CAQE are used in practice for formal verification, model checking, and AI planning. However, systematic testing of these solvers remains an open problem: arbitrary modifications to a QBF formula typically change its truth value, so the correct solver output after a modification is no longer known.

This thesis addresses the problem through *truth-value-preserving transformations*. Such a transformation produces a *mutant* formula that is structurally different from the original but logically equivalent to it. Since both formulas must yield the same solver result by construction, any discrepancy is a definite solver error. Four operators are formally defined and proven to preserve the truth value: tautology insertion, pure literal generation, universal reduction triggering, and blocked clause insertion. Each operator is designed to activate a specific internal solver mechanism.

The operators are implemented in a modular Python tool that parses QDIMACS files, applies transformations, invokes both solvers, and measures runtime and code coverage. An empirical evaluation on 50 benchmark formulas from QBFLIB — 285 valid experiments in total — achieves a 100 % equivalence rate and reveals operator-specific differences in both runtime ratios and coverage deltas. A consistent asymmetry emerges between the two solvers, confirming that truth-value-preserving transformations are an effective and principled method for systematically exercising QBF solver internals.

CONTENTS

1	Introduction	1
1.1	Problem Statement	1
1.2	Approach: Truth-Value-Preserving Transformations	1
1.3	Goals and Contributions	2
2	Background	3
2.1	Quantified Boolean Formulas	3
2.1.1	Relationship to SAT	3
2.1.2	Internal Solver Mechanisms	4
2.2	The QDIMACS Format	4
2.3	QBF Solvers	5
2.3.1	DepQBF	5
2.3.2	CAQE	6
2.4	Code Coverage with gcov	6
2.5	Truth-Value-Preserving Transformations	7
3	Transformation Operators	8
3.1	Tautology Insertion	8
3.1.1	Definition	8
3.1.2	Proof of Truth-Value Preservation	8
3.1.3	Expected Effect on Solver	9
3.2	Pure Literal Generator	9
3.2.1	Definition	9
3.2.2	Proof of Truth-Value Preservation	10
3.2.3	Expected Effect on Solver	10
3.3	Universal Reduction Trigger	11
3.3.1	Definition	11
3.3.2	Proof of Truth-Value Preservation	11
3.3.3	Expected Effect on Solver	12
3.4	Blocked Clause Insertion	12
3.4.1	Definition	12
3.4.2	Proof of Truth-Value Preservation	13
3.4.3	Expected Effect on Solver	13
3.5	Summary	13

4	Implementation	14
4.1	Architecture Overview	15
4.2	Parser	15
4.3	Operator Design	16
4.4	Solver Integration	17
4.4.1	SolverTimer and SolverRunner	17
4.4.2	Equivalence Verification	18
4.5	Coverage Analysis	19
4.6	Pipeline and Result Persistence	20
4.7	Command-Line Interface	20
5	Evaluation	22
5.1	Experimental Setup	22
5.1.1	Benchmark Formulas	22
5.1.2	Operators and Variants	23
5.1.3	Measurement Parameters	23
5.1.4	Equivalence Verification	23
5.2	Runtime Analysis	25
5.2.1	Runtime Ratio per Operator — DepQBF	25
5.2.2	Runtime Ratio per Operator — CAQE	25
5.2.3	Solver Comparison: DepQBF vs. CAQE	26
5.2.4	Interpretation	26
5.3	Coverage Analysis	27
5.3.1	Line Coverage Delta	28
5.3.2	Branch Coverage Delta	28
5.3.3	Interpretation	28
5.4	Secondary Analysis: Coverage Change vs. Runtime Effect	30
5.5	Summary	32
6	Conclusion	34
6.1	Summary of Contributions	34
6.2	Limitations	34
6.3	Future Work	35

LIST OF FIGURES

4.1	Class diagram of the QBF Transformer tool.	14
5.1	Equivalence rate per transformation operator. The dashed green line marks the 100% target. Bar labels show the equivalence rate and sample count n . Operators with $n < 48$ were not applicable to all benchmark formulas (see text).	24
5.2	Runtime ratio (mutant / original) per operator for DepQBF. The dashed red line marks ratio = 1.0.	25
5.3	Runtime ratio (mutant / original) per operator for CAQE. The dashed red line marks ratio = 1.0.	26
5.4	Median runtime ratio comparison between DepQBF and CAQE per operator.	27
5.5	Line coverage delta (Δ line coverage in percentage points, pp) per operator. Bars show the median; error bars show the interquartile range (IQR). Positive values indicate the mutant activates additional source lines in DepQBF.	28
5.6	Branch coverage delta (Δ branch coverage in percentage points, pp) per operator. Bars show the median; error bars show the interquartile range (IQR). Positive values indicate the mutant causes additional conditional branches to be taken in DepQBF.	29
5.7	Coverage delta (pp) vs. runtime ratio for DepQBF. The dashed red line marks ratio = 1.0.	30
5.8	Coverage delta (pp) vs. runtime ratio for CAQE. The dashed red line marks ratio = 1.0.	31
5.9	Coverage delta (pp) vs. runtime ratio for DepQBF, grouped by operator family. The dashed coloured line shows a linear fit with slope m ; the dotted red line marks ratio = 1.0.	31

LIST OF TABLES

3.1	Overview of the four transformation operators.	13
5.1	Aggregated results per transformation operator (n = number of valid experiments; ratios and coverage deltas are medians across formulas).	32

1.1 Problem Statement

Quantified Boolean Formulas (QBF) are a powerful extension of classical propositional logic. Unlike ordinary Boolean formulas, QBF allows the use of existential and universal quantifiers over Boolean variables. This expressiveness enables complex decision problems from formal verification, model checking, and AI planning to be represented precisely and compactly. The decision problem for QBF — is a given quantified formula true or false? — is PSPACE-complete and therefore strictly more expressive than the NP-complete SAT problem.

Specialized tools called QBF solvers exist to solve QBF instances. Prominent examples are DepQBF [1] and CAQE [2], which are based on different algorithmic foundations. Despite their practical importance, the systematic testing of these solvers remains an open problem: unlike classical SAT solvers, there are no established methods to deliberately activate specific internal code paths of a QBF solver and measure their correctness and efficiency.

The fundamental challenge lies in the nature of testing itself: if a QBF formula is modified arbitrarily, its truth value typically changes as well — meaning it is no longer known whether the solver returns the correct result. What is needed is a method that deliberately alters the formula's structure while guaranteeing that the correct solver result remains known.

1.2 Approach: Truth-Value-Preserving Transformations

This thesis addresses the problem described above through the use of truth-value-preserving transformations. Such a transformation modifies a QBF formula structurally — for example by adding a clause, generating a pure literal, or inserting a blocked clause — while formally preserving the truth value of the formula. The result is a so-called *mutant*: a new formula

instance that has the same truth value as the original, but causes the solver to follow a different internal computation path.

This approach is inspired by the concept of metamorphic testing [3]: instead of directly verifying correct output behavior — which would be difficult for QBF without a known solution — relations between inputs are exploited. Since the original and the mutant must share the same truth value, any divergence in the solver’s result can be identified as a fault. At the same time, the structural differences between original and mutant make it possible to deliberately activate specific internal solver mechanisms and measure their effect on runtime and code coverage.

Four transformation operators form the core of this approach, each targeting a different internal QBF mechanism: tautology insertion, pure literal generation, universal reduction triggering, and blocked clause insertion. Each operator is formally proven to be truth-value-preserving and is specifically constructed to activate a particular internal solver code path.

1.3 Goals and Contributions

The central goal of this bachelor’s thesis is the development of a Python tool that automatically transforms QBF formulas in QDIMACS format, solves the transformed formulas using DepQBF and CAQE, and systematically measures runtime and code coverage. On this basis, the thesis investigates how structural properties of a QBF formula affect the behavior of both solvers.

The concrete contributions of this thesis are:

- The formal description and justification of four truth-value-preserving transformation operators for QBF.
- The implementation of a modularly structured Python tool (*QBF Transformer*) that integrates a parser, operators, solver integration, and analysis in a single pipeline.
- An empirical evaluation over benchmark formulas that reports runtime ratios and code coverage deltas for each operator and both solvers.
- A comparative analysis between DepQBF and CAQE showing which transformation effects are solver-independent and which are solver-specific.

2.1 Quantified Boolean Formulas

A *Quantified Boolean Formula* (QBF) is an extension of propositional logic in which Boolean variables may be bound by quantifiers. Formally, a QBF in *prenex normal form* consists of two parts: a *quantifier prefix* and a *matrix*.

The quantifier prefix is a sequence of quantifier blocks of the form

$$Q_1 X_1 Q_2 X_2 \cdots Q_k X_k$$

where each $Q_i \in \{\exists, \forall\}$ is either an existential or a universal quantifier, and each X_i is a non-empty, pairwise disjoint set of Boolean variables. The matrix φ is a propositional formula over all variables $X_1 \cup \cdots \cup X_k$, typically given in conjunctive normal form (CNF) as a conjunction of clauses.

The truth value of a QBF is defined recursively: an existentially quantified variable $x \in X_i$ is *true* if there exists an assignment to x that makes the remaining formula true; a universally quantified variable $u \in X_i$ must satisfy the formula *for all* assignments to u .

Example 2.1. The formula $\forall x_1 \exists x_2 x_3 \forall x_4: (x_1 \vee x_2) \wedge (\neg x_2 \vee x_3 \vee x_4)$ has a universal prefix block $\{x_1, x_4\}$ and an existential block $\{x_2, x_3\}$. The formula is true if the existential player can satisfy the matrix for every assignment to x_1 and x_4 .

2.1.1 Relationship to SAT

The Boolean Satisfiability problem (SAT) asks whether a propositional formula has at least one satisfying assignment. SAT is NP-complete [4]. QBF strictly generalises SAT: a propositional formula φ is equivalent to the QBF $\exists X: \varphi$, where X is the set of all variables. The decision

problem for QBF is PSPACE-complete [5], meaning it is believed to be strictly harder than NP. This additional expressiveness makes QBF a natural fit for problems that require reasoning about all possible environments, such as reactive synthesis, model checking, and adversarial planning.

2.1.2 Internal Solver Mechanisms

Modern QBF solvers exploit several structural properties of the input formula to prune the search space. Four mechanisms are particularly relevant to this thesis, as the transformation operators defined in Chapter 3 are specifically designed to activate them:

Unit Literal. A clause containing exactly one literal is called a *unit clause*. Unit propagation assigns the literal's variable to the unique satisfying value and removes the clause from the formula, simplifying the search.

Pure Literal. A variable is called a *pure literal* if it appears in the formula with only one polarity (only positive or only negative). In propositional SAT, a pure literal can be assigned to satisfy all clauses in which it appears and those clauses can be removed. In QBF, the rule applies dually: existential pure literals are assigned to satisfy the clauses containing them, while universal pure literals are assigned to the opposite value [1].

Universal Reduction. A universal variable u can be removed from a clause C if no existential variable that appears in C is in a quantifier block to the right of u 's block. This operation, known as *universal reduction* (UR), is a sound simplification rule specific to QBF [6].

Blocked Clause. A clause C is *blocked* with respect to a literal $l \in C$ if, for every clause C' containing $\neg l$, the resolvent $C \otimes_l C'$ is a tautology. Blocked clauses can be added to or removed from a formula without changing its truth value [7].

2.2 The QDIMACS Format

QBF instances are commonly exchanged in the *QDIMACS* format [8], the standard input format for QBF solvers. A QDIMACS file consists of four parts:

1. **Comment lines** (optional): lines beginning with *c*.

2. **Header line:** `p cnf <num_vars> <num_clauses>`, declaring the total number of variables and clauses.
3. **Quantifier prefix:** one line per quantifier block, beginning with a (universal) or e (existential), followed by variable indices and terminated by 0.
4. **Matrix:** one clause per line, as a space-separated list of signed integer literals terminated by 0. A positive integer i represents the positive literal of variable i ; a negative integer $-i$ represents its negation.

Listing 2.1 shows a minimal QDIMACS file encoding the formula $\forall x_1 \exists x_2 x_3: (x_1 \vee x_2) \wedge (\neg x_2 \vee x_3)$.

Listing 2.1: A minimal QDIMACS example.

```
c Example formula
p cnf 3 2
a 1 0
e 2 3 0
1 2 0
-2 3 0
```

The `QDIMACSParser` implemented in this project reads such files line by line, validates the header, separates the prefix from the matrix, and constructs a `QBFFormula` object. The parser raises a `ParseError` with file name and line number on any format violation.

2.3 QBF Solvers

2.3.1 DepQBF

DepQBF [1] is a search-based QBF solver written in C. It extends the DPLL procedure to QBF by maintaining a *dependency scheme* that tracks which existential variables depend on which universal variables. This dependency information allows DepQBF to apply universal reduction more aggressively than a naive left-to-right quantifier order would permit. DepQBF serves as

the primary solver in this thesis because its C source code is instrumented with `gcov` flags, enabling fine-grained code coverage measurement.

2.3.2 CAQE

CAQE [2] is a certifying QBF solver written in Rust. It is based on a *clausal abstraction* algorithm that iteratively refines an abstraction of the formula by solving a sequence of SAT sub-problems. Unlike DepQBF, CAQE produces a certificate of its result (a strategy for the winning player). CAQE is used as a second, independent solver to verify that observed runtime differences are not artifacts of a single solver's implementation.

Both solvers follow the QDIMACS exit code convention: exit code 10 indicates the formula is true (satisfiable from the existential player's perspective), exit code 20 indicates false (unsatisfiable).

2.4 Code Coverage with `gcov`

`gcov` is a code coverage tool distributed with GCC. It operates in two phases. First, the target program (here: DepQBF) is compiled with the flags `-fprofile-arcs -ftest-coverage`. This instrumentation step produces `.gcno` files containing the static control-flow structure of the compiled code. Second, each execution of the instrumented binary writes `.gcda` files recording how many times each basic block was executed. Invoking `gcov -b -c` on the source files then combines `.gcno` and `.gcda` to produce a human-readable report of the form:

Lines executed: 40.32% of 3521 Branches executed: 35.18% of 8742

Two metrics are collected per experiment:

Line coverage. The percentage of source lines executed at least once. A line corresponds to one or more machine instructions.

Branch coverage. The percentage of conditional branches (e.g. `if/else`, loop conditions) that were taken at least once in each direction. Branch coverage is strictly finer than line coverage: a line with an `if` statement contributes two branches.

In the experimental pipeline, coverage counters are reset before each solver run by deleting all `.gcda` files. After the run, `gcov` is invoked and its output is parsed by the `CoverageAnalyzer` class. The *coverage delta* Δ between a mutant and its original is then defined as:

$$\Delta_{\text{line}} = \text{line_coverage}(\text{mutant}) - \text{line_coverage}(\text{original})$$

and analogously for branch coverage. A positive delta indicates that the transformed formula caused the solver to execute additional code paths.

2.5 Truth-Value-Preserving Transformations

A transformation T applied to a QBF formula φ is called *truth-value-preserving* if

$$\varphi \text{ is true} \iff T(\varphi) \text{ is true.}$$

The transformed formula $T(\varphi)$ is referred to as a *mutant*. Truth-value-preserving transformations are useful for testing because the correct output of the solver on the mutant is known by construction: it must equal the output on the original. Any deviation is a definite solver error. At the same time, because the structure of the formula changes, the solver may follow entirely different internal code paths, potentially exercising functionality that the original formula does not reach. This combination of correctness oracle and structural diversity is what makes truth-value-preserving transformations an effective systematic testing methodology for QBF solvers.

This chapter describes the four truth-value-preserving transformation operators implemented in the QBF Transformer tool. For each operator we give a formal definition, a proof of truth-value preservation, and a description of the expected effect on solver behavior.

All operators follow the same contract: given an input formula φ , the operator produces a *mutant* φ' such that φ is true if and only if φ' is true. The original formula object is never modified; the operator always works on a deep copy.

3.1 Tautology Insertion

3.1.1 Definition

Let φ be a QBF formula and let x be an existentially quantified variable in φ . The *tautology insertion* operator produces the mutant

$$\varphi' = \varphi \wedge (x \vee \neg x).$$

The clause $(x \vee \neg x)$ is called a *tautology clause* because it evaluates to true under every possible assignment to x .

3.1.2 Proof of Truth-Value Preservation

Proof. Let φ be any QBF formula and x any Boolean variable. The clause $(x \vee \neg x)$ is a tautology, i.e. it is true for every assignment to x . Therefore:

$$\varphi \wedge (x \vee \neg x) \equiv \varphi \wedge \top \equiv \varphi.$$

The truth value of φ' is identical to that of φ under every quantifier assignment. □

3.1.3 Expected Effect on Solver

The tautology clause adds one syntactic clause to the formula without adding any logical constraint. Both DepQBF and CAQE must parse and process the additional clause during preprocessing. Because the clause contains a complementary literal pair $(x, \neg x)$, a solver that performs tautology detection during clause simplification will immediately recognize and discard it. The expected runtime overhead is therefore small (ratio close to 1.0 and slightly below), and the coverage delta is expected to be low, as no fundamentally new solver code paths are activated.

Listing 3.1: Tautology insertion example.

```
% Original (3 variables, 2 clauses)
p cnf 3 2
a 1 0
e 2 3 0
1 -2 0
-2 3 0

% Mutant: one tautology clause added for variable 2
p cnf 3 3
a 1 0
e 2 3 0
1 -2 0
-2 3 0
2 -2 0
```

3.2 Pure Literal Generator

3.2.1 Definition

A variable x is a *pure literal* in a formula φ if it appears in φ with only one polarity: either exclusively as x (positive pure literal) or exclusively as $\neg x$ (negative pure literal), but never both.

The *pure literal generator* operator identifies an existentially quantified variable x that is already a pure literal in φ and applies *Pure Literal Elimination* (PLE) to produce the mutant φ' :

- If x is a positive pure literal: set $x = \top$. Remove all clauses containing x (they are satisfied). Remove the variable x from the quantifier prefix.
- If x is a negative pure literal: set $x = \perp$. Remove all clauses containing $\neg x$ (they are satisfied). Remove x from the prefix.

3.2.2 Proof of Truth-Value Preservation

Proof. Let x be an existentially quantified positive pure literal in φ , i.e. $\neg x$ does not appear in any clause of φ . Because $\neg x$ is absent, assigning $x = \top$ satisfies every clause containing x and leaves all other clauses unchanged. No clause is falsified by this assignment. The existential player can always choose $x = \top$, so the truth value of φ is not affected by the elimination. The argument for a negative pure literal is symmetric. $\square$

Remark. The operator only applies PLE to variables that are *already* pure literals in the original formula. Artificially making a variable pure by removing occurrences of one polarity would not be truth-value-preserving in general.

3.2.3 Expected Effect on Solver

Pure literal elimination removes clauses from the formula, making the mutant strictly smaller than the original. The solver is expected to solve the mutant faster (runtime ratio < 1.0). Because the mutant is smaller, the solver traverses fewer code paths overall, resulting in a negative coverage delta relative to the original run.

3.3 Universal Reduction Trigger

3.3.1 Definition

Universal Reduction (UR) is a simplification rule specific to QBF [6]: a universal literal u may be removed from a clause C if no existential variable in C lies at a deeper quantifier level than u 's block. Formally, let $\text{level}(v)$ denote the index of the quantifier block containing variable v in the prefix, with level 0 corresponding to the outermost block. Then u is removable from C if:

$$\forall e \in C, e \text{ existential} \Rightarrow \text{level}(e) < \text{level}(u).$$

The *universal reduction trigger* operator inserts the clause

$$C_{\text{UR}} = [u, e, \neg e]$$

where u is a universal variable and e is an existential variable with $\text{level}(u) < \text{level}(e)$, i.e. u lies at an outer level than e . This level configuration is chosen so that the solver, when inspecting the clause, has to consult the same quantifier-level and clause-simplification machinery that universal reduction relies on, even though the clause is ultimately discarded as a tautology.

3.3.2 Proof of Truth-Value Preservation

Proof. The clause $[u, e, \neg e]$ is a tautology for every assignment to e :

$$[u, e, \neg e] = \begin{cases} [u, \top, \perp] = \top & \text{if } e = \top, \\ [u, \perp, \top] = \top & \text{if } e = \perp. \end{cases}$$

The clause evaluates to true regardless of the value of u or e . Adding a tautological clause to any formula preserves its truth value:

$$\varphi \equiv \varphi \wedge \top \equiv \varphi \wedge [u, e, \neg e].$$

□

Remark on a prior incorrect design. An earlier version of this operator inserted the clause $[u, e]$ instead of $[u, e, \neg e]$. This is *not* truth-value-preserving: if the universal player assigns $u = \perp$, the clause becomes a unit clause $[e]$, which may force $e = \top$ and create a conflict with existing clauses, potentially turning a satisfiable formula into an unsatisfiable one. The tautological form $[u, e, \neg e]$ avoids this issue entirely.

3.3.3 Expected Effect on Solver

When the solver encounters C_{UR} , it must inspect the quantifier levels of u and e during its standard simplification phase: the clause contains a universal literal whose level is outer to that of the existential pair $e, \neg e$. Even though the clause is ultimately discarded as a tautology (because of the pair $[e, \neg e]$), the solver traverses the same level-checking and clause-simplification code that is also used during universal reduction. The runtime overhead is therefore expected to be small, while the coverage delta should be slightly positive — reflecting that some preprocessing code paths are reached that the original formula does not exercise.

3.4 Blocked Clause Insertion

3.4.1 Definition

Let C be a clause and $l \in C$ a literal. C is *blocked* with respect to l if, for every clause D in the formula that contains $\neg l$, the resolvent $C \otimes_l D = (C \setminus \{l\}) \cup (D \setminus \{\neg l\})$ is a tautology [7].

The *blocked clause insertion* operator constructs such a clause C and adds it to φ . The simplest construction arises when no clause in φ contains $\neg l$: in this case the condition holds vacuously for every clause, and the single-literal clause $[l]$ is trivially blocked. More generally, the operator selects an existential literal l and builds C by appending literals that ensure every resolvent with a $\neg l$ -clause is a tautology.

3.4.2 Proof of Truth-Value Preservation

Proof. This follows directly from the Blocked Clause Addition (BCA) theorem [9, 7]: if C is blocked with respect to some literal $l \in C$, then

$$\varphi \equiv \varphi \wedge C.$$

Intuitively, a blocked clause can always be satisfied by setting $l = \top$ without falsifying any other clause, because every clause D containing $\neg l$ already contains a tautological resolvent pair with C . The truth value of φ is therefore unchanged. $\square$

3.4.3 Expected Effect on Solver

Both DepQBF and CAQE implement blocked clause detection as part of their preprocessing phase. Inserting a blocked clause forces the solver to execute the BCE/BCA detection code paths, which increases coverage for the corresponding solver modules. The runtime effect depends on the size of the formula and the cost of BCE detection; in general, the ratio is expected to be close to 1.0 with a moderate positive coverage delta.

3.5 Summary

Table 3.1 summarises the four operators, their structural change to the formula, the basis of truth-value preservation, and the primary solver mechanism each is designed to activate.

Table 3.1: Overview of the four transformation operators.

Operator	Structural change	Preservation basis	Target mechanism
Tautology Insertion	Adds clause $[x, \neg x]$	Tautology ($\equiv \top$)	Tautology detection
Pure Literal Generator	Removes clauses, shrinks prefix	PLE soundness	Pure Literal Elimination
Universal Reduction Trigger	Adds clause $[u, e, \neg e]$	Tautology ($\equiv \top$)	Universal Reduction
Blocked Clause Insertion	Adds blocked clause C	BCA theorem	Blocked Clause Elimination

CHAPTER IMPLEMENTATION

4

This chapter describes the design and implementation of the *QBF Transformer* tool. The tool is written in Python 3 and implements the five-step experimental pipeline introduced in Chapter 1. The architecture is organized into five layers, each with a clearly defined responsibility. Figure 4.1 shows the class diagram of the complete system.

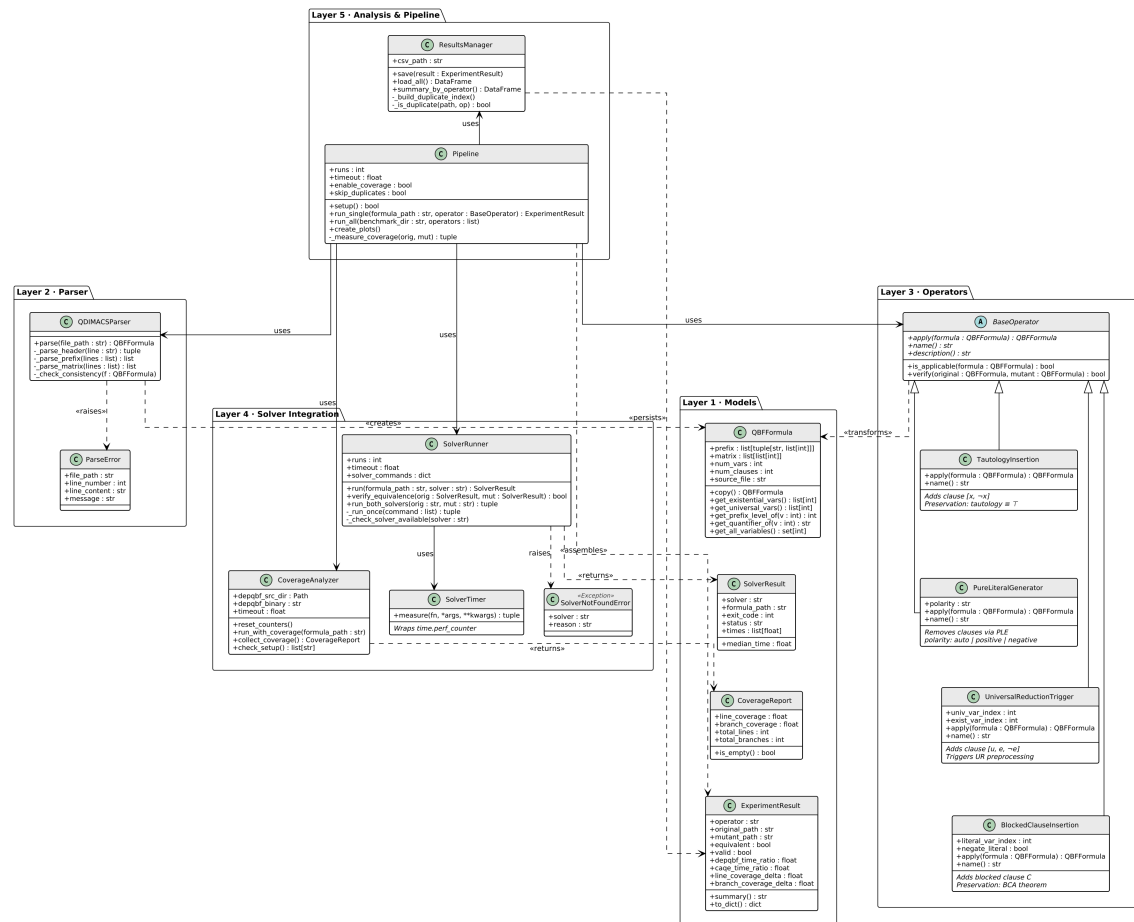


Figure 4.1: Class diagram of the QBF Transformer tool.

4.1 Architecture Overview

The system is structured into five layers that correspond directly to the five steps of the experimental pipeline:

1. **Models** — data classes that represent the domain objects: `QBFFormula`, `SolverResult`, `CoverageReport`, and `ExperimentResult`.
2. **Parser** — the `QDIMACSParser` and `ParseError` classes, which read `.qdimacs` files and validate their structure.
3. **Operators** — the `BaseOperator` abstract class and its four concrete subclasses, one per transformation.
4. **Solver integration** — `SolverTimer`, `SolverRunner`, and `CoverageAnalyzer`, which invoke the external solvers and collect measurements.
5. **Analysis and pipeline** — `ResultsManager`, the `Pipeline` class in `main.py`, which orchestrate the full experiment loop and persist results, and a Jupyter notebook (`qb_f_solver_analysis_notebook.ipynb`), located in the project root directory, for interactive plot generation and data analysis.

The dependency direction is strictly top-down: each layer uses only the layer below it, and no circular dependencies exist between modules. An additional utility script, `filter_benchmarks.py`, is provided outside the main pipeline to select suitable benchmark formulas from the QBFLIB repository by filtering on variable count, clause count, and prefix-block count; it was used to produce the 50 benchmark formulas evaluated in Chapter 5.

4.2 Parser

The `QDIMACSParser` class reads a `.qdimacs` file line by line and constructs a `QBFFormula` object. Parsing proceeds in four stages:

1. **Comment and blank line removal.** Lines beginning with `c` and empty lines are discarded. All remaining lines are kept with their original line numbers for error reporting.

2. **Header parsing.** The first non-comment line must match the pattern `p cnf <num_vars> <num_clauses>`. Both integers are validated to be positive.
3. **Prefix parsing.** Consecutive lines beginning with `a` (universal) or `e` (existential) are parsed into a list of `(quantifier, variables)` tuples. Each line must be terminated by `0`.
4. **Matrix parsing.** The remaining lines are parsed as clauses. Each clause is a list of non-zero signed integers terminated by `0`.

After parsing, a consistency check verifies that the number of parsed clauses matches `num_clauses` from the header, and that every variable referenced in the matrix appears in the prefix. Any violation raises a `ParseError` that includes the file path, line number, and offending line content, enabling precise diagnosis.

4.3 Operator Design

All four transformation operators share a common interface defined by the abstract base class `BaseOperator`. The class enforces the *Template Method* pattern: subclasses must implement three abstract methods and may override two optional ones.

`apply(formula)` Applies the transformation and returns a new `QBFFormula` object. The original formula must never be modified; the method always calls `formula.copy()` before making any changes.

`name()` Returns a unique, machine-readable identifier string (e.g. `"tautology_insertion"`), used as a column value in the CSV output and as a file-name suffix for mutants.

`description()` Returns a human-readable explanation of the transformation.

`is_applicable(formula)` (optional) Returns `True` if the preconditions for the operator are met. For example, `UniversalReductionTrigger` requires at least one universal variable u and one existential variable e with $\text{level}(u) < \text{level}(e)$ (see Section 3.3). If `is_applicable` returns `False`, the pipeline skips the experiment rather than raising an error.

`verify(original, mutant)` (optional) Performs a post-hoc structural sanity check. This is not a full equivalence proof — that role is played by the solvers — but a fast check that the transformation was carried out as intended (e.g. the mutant has exactly one more clause than the original).

The eight operator instances registered in `main.py` are:

- `TautologyInsertion()`
- `PureLiteralGenerator(polarity="auto")`, `PureLiteralGenerator(polarity="positive")`, `PureLiteralGenerator(polarity="negative")`
- `UniversalReductionTrigger()`
- `BlockedClauseInsertion()`, `BlockedClauseInsertion(negate_literal=True)`, `BlockedClauseInsertion(literal_var_index=1)`

Each variant has a distinct `name()` return value, ensuring that results from different configurations of the same operator are stored as separate rows in the CSV. For `BlockedClauseInsertion`, the suffix `_neg` is appended when `negate_literal=True`, and the suffix `_v{index}` is appended when `literal_var_index` is non-zero; thus `BlockedClauseInsertion(literal_var_index=1)` produces the name `blocked_clause_insertion_v1`.

4.4 Solver Integration

4.4.1 SolverTimer and SolverRunner

Runtime measurement is handled by `SolverTimer`, which wraps each solver invocation with `time.perf_counter`. This clock is monotonic and unaffected by system time corrections, making it suitable for high-resolution process timing.

`SolverRunner` invokes the external solver binaries via Python's `subprocess` module. Each solver is called with the path to the formula file as a command-line argument:

Listing 4.1: Solver invocation commands.

```
# DepQBF
depqbf --qdo <formula.qdimacs>

# CAQE
caqe <formula.qdimacs>
```

The solver's exit code is interpreted according to the QDIMACS convention: exit code 10 maps to SAT, exit code 20 maps to UNSAT, and any other exit code (including timeout) is recorded as ERROR. Each formula is solved once per experiment. Since the primary research focus is on correctness (equivalence preservation) and code coverage rather than on precise performance benchmarking, a single run is sufficient to detect operator-specific differences in solver behavior. The reported runtime is the wall-clock time of that run.

The *runtime ratio* for a single experiment is defined as:

$$\text{ratio} = \frac{\text{time}(\text{mutant})}{\text{time}(\text{original})}.$$

A ratio greater than 1.0 indicates that the mutant takes longer to solve; a ratio less than 1.0 indicates that the mutant is solved faster.

4.4.2 Equivalence Verification

After both the original and the mutant have been solved, SolverRunner checks that the results are consistent. An experiment is marked `equivalent = True` if and only if the status of the original and the status of the mutant are identical (both SAT or both UNSAT) for **DepQBF**. The CAQE results are recorded in the CSV and can be inspected post-hoc, but are not used as part of the formal equivalence flag. Any discrepancy in the DepQBF results is flagged as a truth-value violation and excluded from the runtime and coverage analysis.

4.5 Coverage Analysis

Code coverage is measured using `gcov`, which requires DepQBF to be compiled with GCC instrumentation flags:

Listing 4.2: DepQBF instrumented build command.

```
make CFLAGS="-fprofile-arcs -ftest-coverage -O0"
```

The `CoverageAnalyzer` class implements a three-step measurement protocol for each formula:

1. **Reset.** All `.gcda` files in the DepQBF source directory are deleted. This clears the execution counters from any previous run.
2. **Execute.** DepQBF is invoked once on the formula via subprocess. The instrumented binary writes `.gcda` files recording which basic blocks were executed and how many times.
3. **Collect.** `gcov` is called on the DepQBF source files. Its standard output contains lines of the form `Lines executed: X% of N` and `Branches executed: Y% of M`, which are extracted by the `CoverageAnalyzer` parser and stored in a `CoverageReport` object.

This protocol is executed twice per experiment: once for the original formula and once for the mutant. The coverage delta is then computed as:

$$\Delta_{\text{line}} = \text{line_coverage}(\text{mutant}) - \text{line_coverage}(\text{original}),$$

and analogously for branch coverage. A positive delta indicates that the mutant caused the solver to traverse additional code paths.

Platform note. On macOS, Apple Clang does not support `gcov` instrumentation. The instrumented DepQBF binary must therefore be compiled with `gcc-15` installed via Homebrew, and the `gcov` binary must be explicitly set to `gcov-15`.

4.6 Pipeline and Result Persistence

The `Pipeline` class in `main.py` orchestrates the complete experimental loop. Its `run_all()` method iterates over every combination of benchmark formula and operator, calling `run_single()` for each pair. The `run_single()` method executes the five pipeline steps in sequence:

1. Parse the original formula with `QDIMACSParser`.
2. Apply the operator to produce a mutant and write it to disk as a `.qdimacs` file.
3. Run both solvers on original and mutant with `SolverRunner`.
4. Measure coverage for both with `CoverageAnalyzer`.
5. Assemble an `ExperimentResult` and persist it via `ResultsManager`.

`ResultsManager` appends each result immediately to a CSV file using `pandas`. All plots and statistical summaries are produced separately in the Jupyter notebook `qbf_solver_analysis_notebook.ipynb`, which loads the CSV and generates the figures used in Chapter 5. Writing incrementally rather than in batch ensures that no data is lost if the pipeline is interrupted. A duplicate detection mechanism based on an in-memory set (keyed on formula path, operator name, and mutant path) prevents re-running experiments that are already present in the CSV, enabling the pipeline to be resumed after interruption in $O(1)$ per lookup.

The CSV schema contains one row per experiment with the following columns: `operator`, `original_path`, `mutant_path`, `equivalent`, `valid`, solver status and median time for both solvers on both formulas, the runtime ratio for each solver, and line and branch coverage for original, mutant, and their delta. This flat structure is directly consumed by the Jupyter notebook described in Chapter 5.

4.7 Command-Line Interface

The tool is invoked from the command line via `main.py`. The most common usage patterns are:

Listing 4.3: CLI usage examples.

```
# Run all formulas x all operators
python main.py

# Run one specific formula
python main.py --formula benchmarks/formula_001.qdimacs

# Run one specific operator only
python main.py --operator tautology_insertion

# Adjust run count and timeout
python main.py --timeout 30

# Disable coverage measurement (faster)
python main.py --no-coverage

# Generate summary from existing CSV (no new experiments)
python main.py --plots-only
```

Additional configuration arguments control solver binary paths (`--depqbf-src`, `--depqbf-binary`), the output CSV path (`--results-csv`), the benchmark directory (`--benchmark-dir`), and duplicate detection behaviour (`--no-skip-duplicates`, which disables the duplicate check and forces re-running all experiments). By default, the pipeline skips formula-operator combinations that are already present in the CSV, enabling interrupted runs to be resumed without recomputing existing results.

CHAPTER

EVALUATION

5

This chapter presents the experimental results of the QBF Transformer tool. Section 5.1 describes the experimental setup, and Section 5.1.4 verifies the correctness of the transformations via the equivalence rate. Sections 5.2–5.4 present the measurements for runtime, coverage, and the relationship between coverage change and runtime effect. Section 5.5 summarises all results in a single table.

5.1 Experimental Setup

5.1.1 Benchmark Formulas

The experiments were conducted on a set of 50 QBF benchmark formulas in QDIMACS format, drawn from the QBFLIB repository (Basler, Castellini, Jordan-Kaiser, Kronegger-Pfandler-Pichler, Tacchella, Tentrup, and Wintersteiger benchmark families). Formulas were selected using the `filter_benchmarks.py` utility, retaining only instances that satisfy all of the following criteria:

- at most 200 variables,
- at most 1 000 clauses,
- at least two quantifier prefix blocks, and
- at least one universal and at least one existential quantifier block (formulas with only a single quantifier type are excluded as trivial).

A total of 400 experiments were recorded across all operator and formula combinations (50 formulas $\times$ 8 operators). After filtering for operator applicability, 285 experiments were retained for analysis. The equivalence rate and the reasons for exclusion are discussed in Section 5.1.4.

5.1.2 Operators and Variants

Eight operator variants were evaluated, grouped into four families:

- **Tautology Insertion** (1 variant): `tautology_insertion`
- **Pure Literal Generator** (3 variants, differing in which polarity of pure literal is selected): `pure_literal_gen_auto` (any polarity), `pure_literal_gen_pos` (positive only), `pure_literal_gen_neg` (negative only)
- **Universal Reduction Trigger** (1 variant): `universal_reduction_trigger`
- **Blocked Clause Insertion** (3 variants, differing in the blocking-literal selection): `blocked_clause_insertion` (first existential, positive), `blocked_clause_insertion_neg` (first existential, negated), `blocked_clause_insertion_v1` (second existential)

5.1.3 Measurement Parameters

Each formula was solved once per solver per experiment. As the focus of this thesis is correctness verification and coverage analysis rather than fine-grained performance benchmarking, a single run is adequate to capture operator-specific effects. A per-experiment timeout of 60 seconds was applied. Runtime was measured with `time.perf_counter`; code coverage was collected via `gcov` on an instrumented build of DepQBF compiled with `gcc-15`.

5.1.4 Equivalence Verification

Before analysing runtime and coverage, the correctness of each transformation must be verified. An experiment is marked as *equivalent* if and only if DepQBF returns the same result (SAT or UNSAT) for both the original and the mutant formula (see Section 4.4.2). CAQE results were additionally recorded and showed no discrepancies when inspected post-hoc, but are not used as part of the formal equivalence flag.

Of 400 total experiments, 285 (71.25%) produced valid results and were retained for runtime and coverage analysis. The remaining 115 experiments were excluded for two reasons:

- **Operator non-applicability** (99 experiments): the Pure Literal Generator variants could not be applied to formulas that contained no pure literals.
- **Parse errors** (16 experiments): two benchmark files (p10-1.pddl_planlen=0 and p5-5.pddl_planlen=0) contained an empty quantifier block (e 0), which is non-standard QDIMACS and caused a parse error in the tool (2 files $\times$ 8 operators = 16 experiments).

Crucially, all 285 retained experiments achieved a 100% equivalence rate: DepQBF returned identical results (SAT or UNSAT) for every original–mutant pair, confirming that all four transformation operators are correctly implemented and truth-value-preserving in practice.

Figure 5.1 shows the equivalence rate per operator. Operators for which all experiments were retained (Tautology Insertion, Universal Reduction Trigger, and all Blocked Clause variants) show a bar at 100% with $n = 48$. The Pure Literal Generator variants show lower n values (19, 13, 13) reflecting the cases where the operator was not applicable.

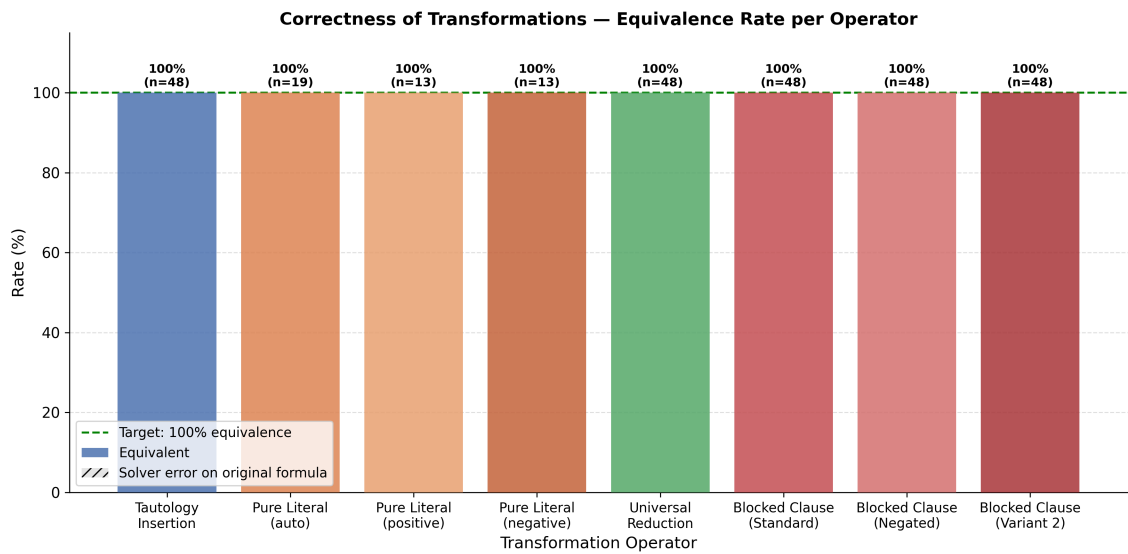


Figure 5.1: Equivalence rate per transformation operator. The dashed green line marks the 100% target. Bar labels show the equivalence rate and sample count n . Operators with $n < 48$ were not applicable to all benchmark formulas (see text).

5.2 Runtime Analysis

The runtime effect of a transformation is quantified by the runtime ratio:

$$\text{ratio} = \frac{\text{median_time}(\text{mutant})}{\text{median_time}(\text{original})}.$$

A ratio of 1.0 indicates no change; values above 1.0 indicate the mutant is slower; values below 1.0 indicate the mutant is faster.

5.2.1 Runtime Ratio per Operator — DepQBF

Figure 5.2 shows the distribution of runtime ratios for DepQBF across all operators.

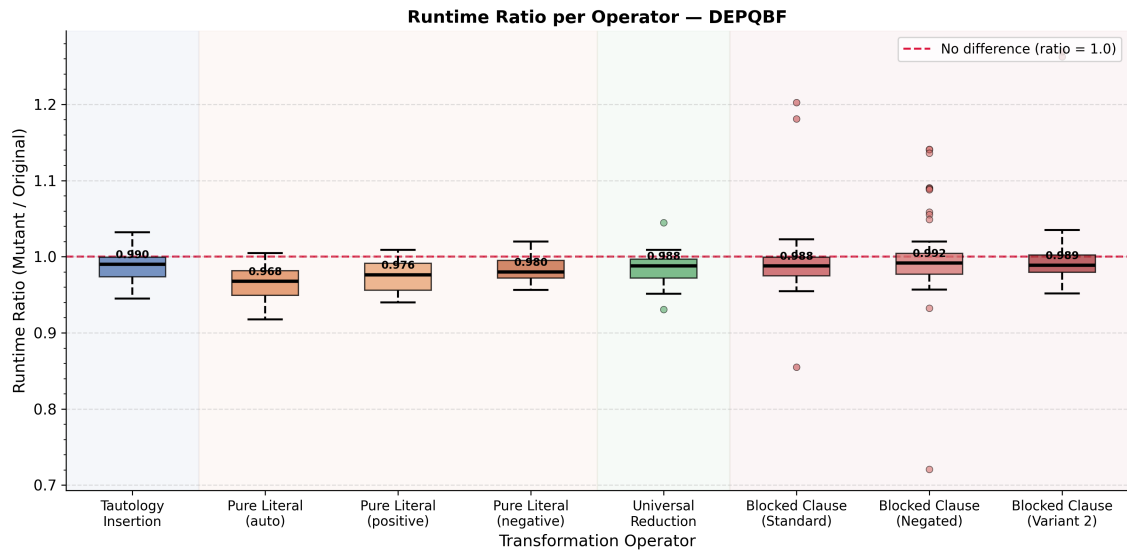


Figure 5.2: Runtime ratio (mutant / original) per operator for DepQBF. The dashed red line marks ratio = 1.0.

5.2.2 Runtime Ratio per Operator — CAQE

Figure 5.3 shows the same distribution for CAQE.

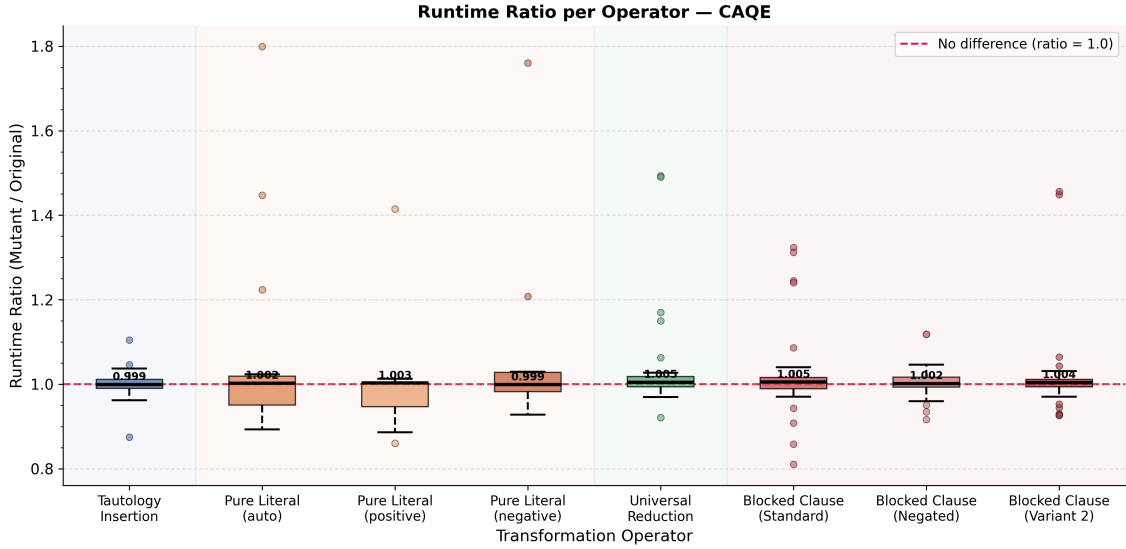


Figure 5.3: Runtime ratio (mutant / original) per operator for CAQE. The dashed red line marks ratio = 1.0.

5.2.3 Solver Comparison: DepQBF vs. CAQE

Figure 5.4 directly compares the median runtime ratio of DepQBF and CAQE for each operator. Operators where both bars are on the same side of 1.0 suggest a general structural effect; operators where the bars diverge suggest a solver-specific implementation effect.

5.2.4 Interpretation

The Tautology Insertion operator produces a median runtime ratio of 0.990 for DepQBF and 0.999 for CAQE, both close to 1.0. The values fall just below 1.0, consistent with measurement noise at this scale, and confirm that adding a trivially discarded clause causes only minimal overhead. The Pure Literal Generator variants consistently show ratios below 1.0 for DepQBF (auto: 0.968, positive: 0.976, negative: 0.980), consistent with the prediction that eliminating clauses reduces the formula size and accelerates DepQBF. For CAQE the corresponding ratios are essentially at 1.0 (auto: 1.002, positive: 1.003, negative: 0.999), suggesting that the size reduction has a much weaker effect on CAQE’s clausal-abstraction strategy than on DepQBF’s search-based procedure. The Universal Reduction Trigger shows a small but notable solver asymmetry: 0.988 for DepQBF (mutant slightly faster) versus 1.005 for CAQE (mutant slightly

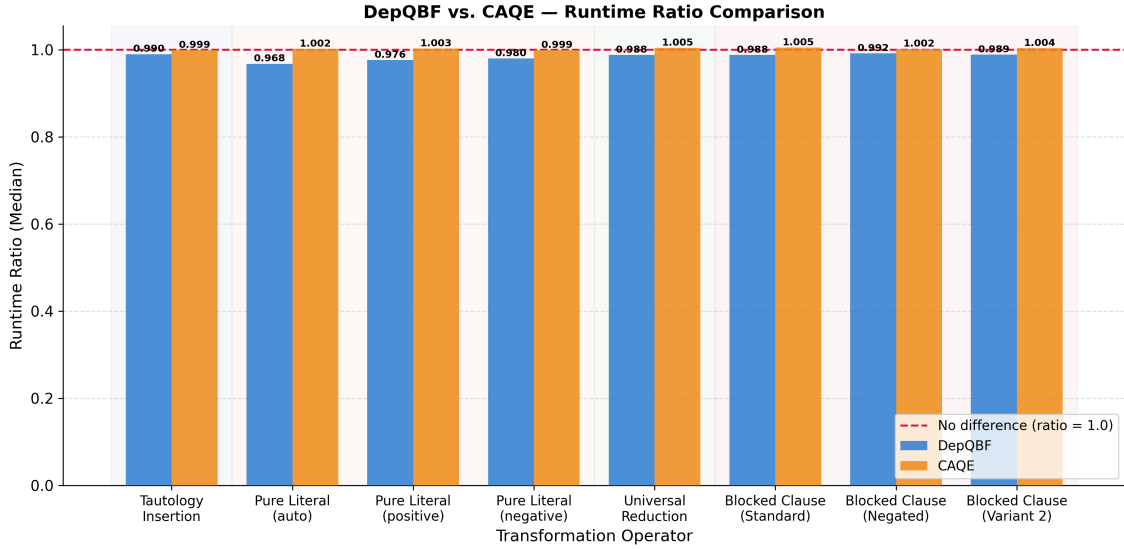


Figure 5.4: Median runtime ratio comparison between DepQBF and CAQE per operator.

slower), suggesting that DepQBF benefits marginally from the inserted clause being simplified away during preprocessing, while CAQE incurs a small overhead from the additional clause. The Blocked Clause Insertion variants show ratios slightly below 1.0 for DepQBF (0.988–0.992), and ratios slightly above 1.0 for CAQE (1.002–1.005), consistent with the same solver asymmetry and the formula-structure dependence of blocked clause detection.

5.3 Coverage Analysis

Code coverage is measured as the percentage of DepQBF source lines and branches executed during solving. The coverage delta Δ is defined as:

$$\Delta = \text{coverage}(\text{mutant}) - \text{coverage}(\text{original}).$$

A positive delta indicates that the mutant formula caused the solver to execute additional code; a negative delta indicates that fewer code paths were reached. Coverage was collected only for DepQBF, as gcov instrumentation is not available for the CAQE binary.

5.3.1 Line Coverage Delta

Figure 5.5 shows the median line coverage delta per operator. Error bars represent the interquartile range (IQR) across all experiments in that operator group.

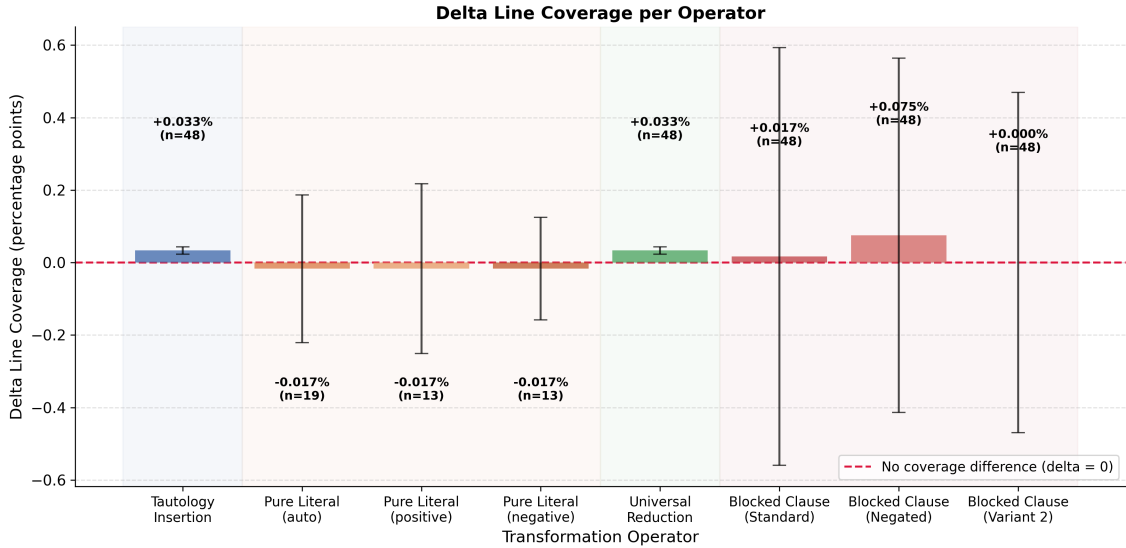


Figure 5.5: Line coverage delta (Δ line coverage in percentage points, pp) per operator. Bars show the median; error bars show the interquartile range (IQR). Positive values indicate the mutant activates additional source lines in DepQBF.

5.3.2 Branch Coverage Delta

Figure 5.6 shows the median branch coverage delta per operator. Branch coverage is strictly finer than line coverage: a single if-statement contributes two branches, so branch deltas tend to be slightly larger in magnitude than the corresponding line deltas.

5.3.3 Interpretation

Tautology Insertion produces a median line coverage delta of +0.033 pp and a branch delta of +0.030 pp. Both values are positive, indicating that the inserted tautological clause ($\ell \vee \neg \ell$) triggers an additional preprocessing code path in DepQBF that is not reached by the original formula.

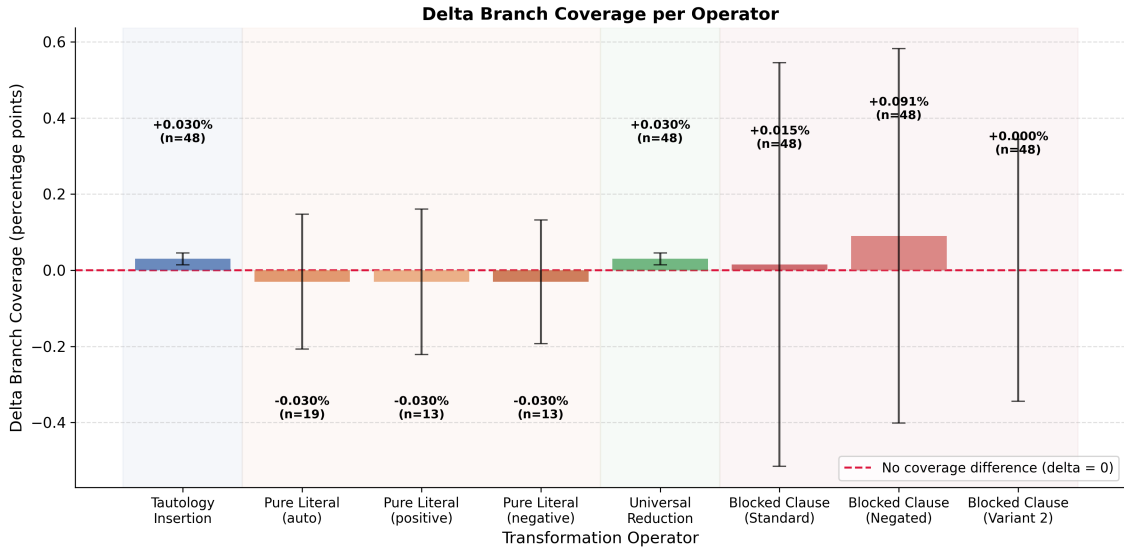


Figure 5.6: Branch coverage delta (Δ branch coverage in percentage points, pp) per operator. Bars show the median; error bars show the interquartile range (IQR). Positive values indicate the mutant causes additional conditional branches to be taken in DepQBF.

Pure Literal Generator (all three variants: auto, positive, negative) consistently produce negative deltas of -0.017 pp (line) and -0.030 pp (branch). This is consistent with the design intent of this operator: by applying Pure Literal Elimination to produce a strictly smaller mutant, the transformed formula traverses fewer solver-internal code paths than the original. All three variants produce identical coverage deltas because they share the same elimination mechanism and differ only in which pure literal is selected.

Universal Reduction Trigger produces a median line delta of $+0.033$ pp and branch delta of $+0.030$ pp, identical to Tautology Insertion. This suggests that the inserted clause $[u, e, \neg e]$ activates the same preprocessing code path as the tautological clause — consistent with the expectation that Universal Reduction is applied during the same preprocessing phase as tautology detection.

Blocked Clause Insertion (Standard variant) produces a median line delta of $+0.017$ pp and branch delta of $+0.015$ pp, approximately half the delta of Tautology Insertion. This suggests partial activation of the blocked clause detection code path.

Blocked Clause (Negated) produces the largest positive delta of all operators: $+0.075$ pp line and $+0.091$ pp branch. The larger delta compared to the standard variant suggests that

negating the blocking literal activates a different, less frequently reached branch within the blocked clause detection routine of DepQBF.

Blocked Clause (Variant 2) produces a delta of exactly 0 pp for both line and branch coverage. The inserted clause is handled by an already-covered code path, meaning no new lines or branches are reached by the mutant.

It is noteworthy that the Blocked Clause (Negated) and Blocked Clause (Standard) variants exhibit large IQR error bars (approximately ± 0.5 pp), reflecting high variability across benchmark formulas. This variability is formula-structure-dependent: blocked clause detection is only triggered when the inserted clause is blocked with respect to the formula's existing clause set, which varies per instance.

5.4 Secondary Analysis: Coverage Change vs. Runtime Effect

This section investigates whether a larger coverage delta is associated with a larger runtime ratio. Figures 5.7 and 5.8 provide an overview across both solvers: each point represents one experiment, with the coverage delta on the x -axis and the runtime ratio on the y -axis. Points are colour-coded by operator family.

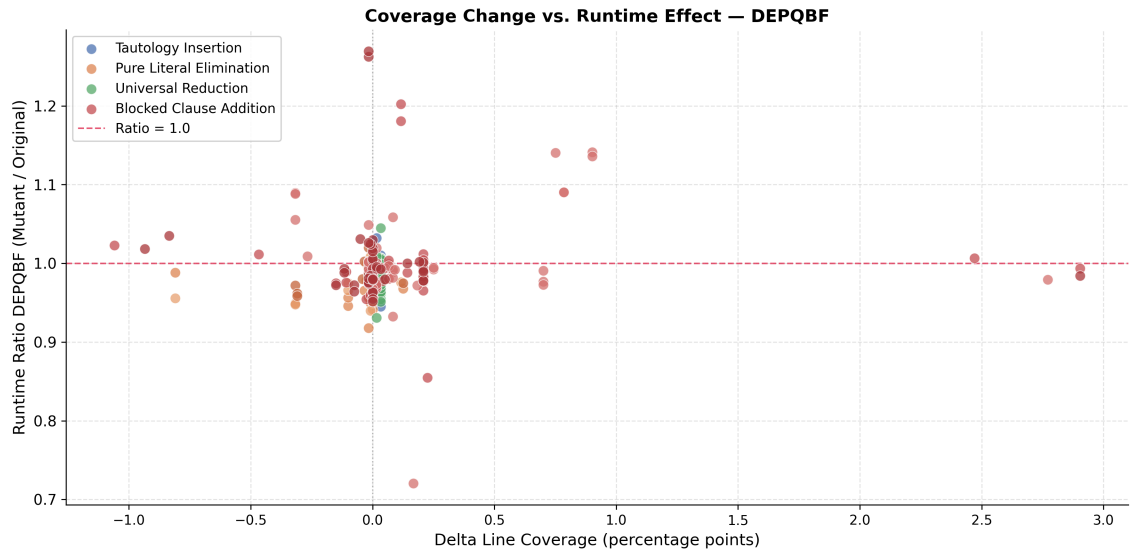


Figure 5.7: Coverage delta (pp) vs. runtime ratio for DepQBF. The dashed red line marks ratio = 1.0.

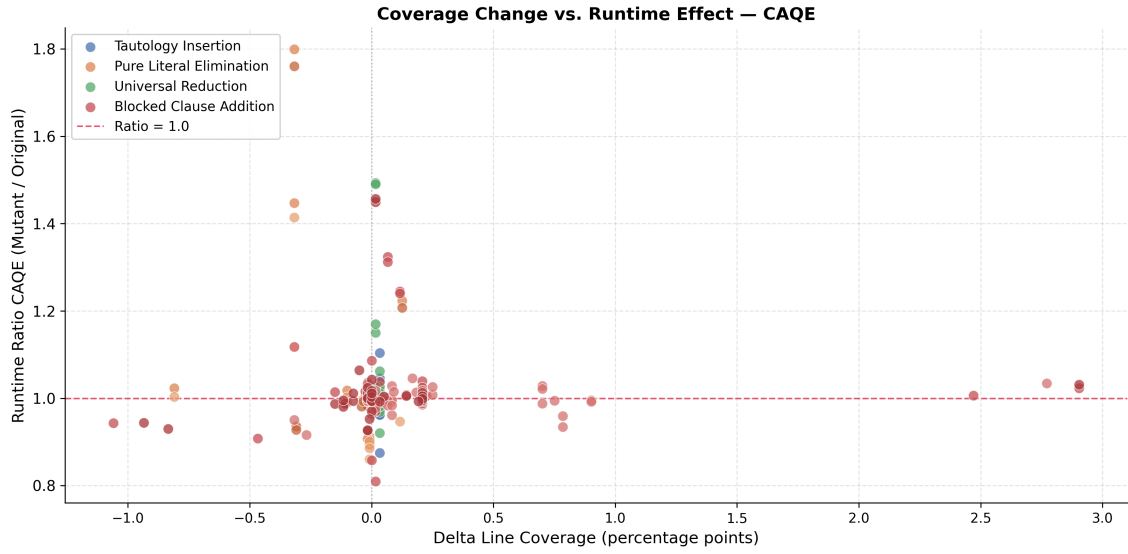


Figure 5.8: Coverage delta (pp) vs. runtime ratio for CAQE. The dashed red line marks ratio = 1.0.

Figure 5.9 then breaks the DepQBF data down by operator family, with a linear trend line with slope m fitted per group.

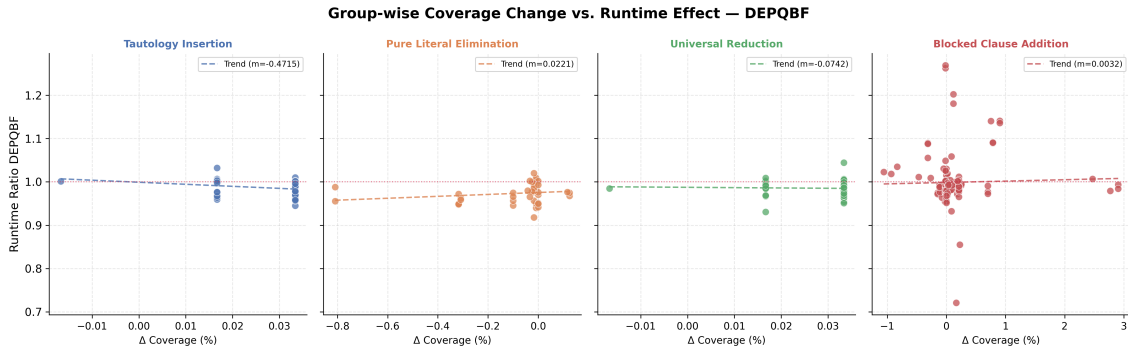


Figure 5.9: Coverage delta (pp) vs. runtime ratio for DepQBF, grouped by operator family. The dashed coloured line shows a linear fit with slope m ; the dotted red line marks ratio = 1.0.

For three of the four operator groups the fitted slope is close to zero ($|m| \leq 0.074$), indicating no meaningful linear relationship between coverage delta and runtime ratio. The exception is **Tautology Insertion** ($m = -0.472$): a larger positive coverage delta is weakly associated with a *lower* runtime ratio, suggesting that reaching additional preprocessing code paths slightly reduces solving time rather than increasing it. However, the Tautology Insertion group spans a

narrow coverage range (approximately -0.01 to $+0.03$ pp) with few distinct delta values, so this slope should not be over-interpreted.

The **Blocked Clause Addition** group exhibits the widest spread on both axes: coverage deltas range from approximately -1 pp to $+3$ pp, and runtime ratios range from roughly 0.75 to 1.25 . Despite this spread, the slope is nearly flat ($m = 0.003$), confirming that coverage delta is not a predictor of runtime change within this group. The large variance is consistent with the formula-structure dependence of blocked clause detection noted in Section 5.3.3.

Overall, the analysis provides no evidence that a larger coverage delta causes a larger runtime effect across any operator group.

5.5 Summary

Table 5.1: Aggregated results per transformation operator (n = number of valid experiments; ratios and coverage deltas are medians across formulas).

Operator	Group	n	Ratio DepQBF	Ratio CAQE	Δ Line (pp)	Δ Branch (pp)
tautology_insertion	Tautology	48	0.990	0.999	+0.033	+0.030
pure_literal_gen_auto	Pure Literal	19	0.968	1.002	-0.017	-0.030
pure_literal_gen_pos	Pure Literal	13	0.976	1.003	-0.017	-0.030
pure_literal_gen_neg	Pure Literal	13	0.980	0.999	-0.017	-0.030
universal_reduction_trigger	Universal Reduction	48	0.988	1.005	+0.033	+0.030
blocked_clause_insertion	Blocked Clause	48	0.988	1.005	+0.017	+0.015
blocked_clause_insertion_neg	Blocked Clause	48	0.992	1.002	+0.075	+0.091
blocked_clause_insertion_v1	Blocked Clause	48	0.989	1.004	+0.000	+0.000

All DepQBF runtime ratios fall within the range 0.968 – 0.992 , confirming that none of the transformations causes measurable overhead for DepQBF. The Pure Literal Generator variants produce the lowest DepQBF ratios (0.968 – 0.980), consistent with the expectation that a smaller mutant formula reduces solving time.

For CAQE, the picture is reversed for several operators: Universal Reduction, Blocked Clause (Standard), and Blocked Clause (Variant 2) all show CAQE ratios slightly above 1.0 (1.004 – 1.005), meaning the mutant is marginally *slower* for CAQE. This solver-specific asymmetry suggests that the two solvers handle the inserted clauses differently during preprocessing: DepQBF appears to simplify them away at a net benefit, while CAQE incurs a small overhead.

Coverage deltas remain small across all operators (below 0.10 pp in all cases), reflecting the limited size of the benchmark formulas. The Blocked Clause (Negated) variant produces

the largest positive delta (+0.075 pp line, +0.091 pp branch), suggesting that negating the blocking literal activates additional preprocessing code paths in DepQBF. The Pure Literal Generator variants consistently produce negative deltas (−0.017 pp line, −0.030 pp branch); the three variants yield identical deltas because they share the same elimination mechanism and differ only in which pure literal is selected. A smaller mutant formula causes the solver to traverse fewer code paths overall.

6.1 Summary of Contributions

This thesis investigated whether truth-value-preserving transformations can be used to systematically test QBF solvers by deliberately activating specific internal mechanisms while preserving the correct solver output as an oracle. To this end, the *QBF Transformer* tool was designed and implemented as a complete experimental pipeline covering parsing, transformation, solving with DepQBF and CAQE, runtime measurement, and gcov coverage analysis.

Four operators were formally defined and proven truth-value-preserving (see Chapter 3): *Tautology Insertion*, *Pure Literal Generator*, *Universal Reduction Trigger*, and *Blocked Clause Insertion*. Each operator targets a distinct internal solver mechanism.

The empirical evaluation over 285 valid experiments achieved a 100% equivalence rate, confirming that all four operators are correctly implemented in practice. The aggregated measurements (Table 5.1) show that the operators produce measurable, operator-specific effects on both runtime and coverage, though the absolute magnitudes are small for the benchmark set used. A consistent asymmetry between the two solvers emerged: DepQBF shows runtime ratios at or below 1.0 across all operators, whereas CAQE shows ratios at or above 1.0 for most operators. This suggests solver-specific differences in preprocessing that warrant further investigation.

6.2 Limitations

Benchmark size. Only 50 formulas from seven QBFLIB families were evaluated. The small absolute coverage deltas (below 0.10 pp) reflect the limited size of the benchmarks rather than the absence of an effect; larger and more diverse sets (e.g. the QBF Gallery) would be needed for statistically robust conclusions.

Coverage measurement limited to DepQBF. gcov is not directly applicable to CAQE's Rust build, so the runtime asymmetry observed for CAQE cannot currently be explained at the code-path level.

Equivalence check via DepQBF only. The equivalence flag relies on DepQBF's output; a systematic DepQBF bug would not be detected by this check alone.

Structural transformations only. All four operators are syntactic. Semantic transformations exploiting dependency schemes or quantifier-alternation depth could potentially activate deeper solver code paths.

6.3 Future Work

Several directions follow naturally from this work. *Larger benchmark sets* from QBFLIB or the QBF Gallery would enable statistically meaningful per-family comparisons. *Additional operators* could target clause learning (Q-resolution), dependency scheme computation, or certificate generation. *Mutation testing* of the solver source itself would quantify the sensitivity of the test method by checking whether injected bugs are detected by the equivalence flag. *Automated operator synthesis* using constraint solving or program synthesis could discover new truth-value-preserving transformations beyond those designed by hand. Finally, *cross-solver coverage* via Rust-native tooling (e.g. cargo-llvm-cov) would allow the DepQBF/CAQE runtime asymmetry to be investigated symmetrically — arguably the most important open question left by this thesis.

BIBLIOGRAPHY

- [1] Florian Lonsing and Armin Biere. “DepQBF: A Dependency-Aware QBF Solver”. In: *Journal on Satisfiability, Boolean Modelling and Computation* 7 (2010) (cit. on pp. 1, 4, 5).
- [2] Markus N. Rabe and Leander Tentrup. “CAQE: A Certifying QBF Solver”. In: *Proceedings of FMCAD*. 2015 (cit. on pp. 1, 6).
- [3] Tsong Yueh Chen et al. “Metamorphic Testing: A Review of Challenges and Opportunities”. In: *ACM Computing Surveys* 51.1 (2018), 4:1–4:27 (cit. on p. 2).
- [4] Stephen A. Cook. “The Complexity of Theorem Proving Procedures”. In: *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing (STOC)*. 1971, pp. 151–158 (cit. on p. 3).
- [5] Larry J. Stockmeyer and Albert R. Meyer. “Word Problems Requiring Exponential Time”. In: *Proceedings of STOC*. 1973, pp. 1–9 (cit. on p. 4).
- [6] Hans Kleine Büning, Marek Karpinski, and Andreas Flögel. “Resolution for Quantified Boolean Formulas”. In: *Information and Computation* 117.1 (1995), pp. 12–18 (cit. on pp. 4, 11).
- [7] Armin Biere, Florian Lonsing, and Maximilian Seidl. “Blocked Clause Elimination for QBF”. In: *Proceedings of CADE-23*. Vol. 6803. LNCS. 2011, pp. 101–115 (cit. on pp. 4, 12, 13).
- [8] QBFLIB. *QDIMACS Standard Version 1.1*. 2005. URL: <https://www.qbflib.org/qdimacs.html> (cit. on p. 4).
- [9] Matti Järvisalo, Armin Biere, and Marijn Heule. “Blocked Clause Elimination”. In: *Proceedings of TACAS*. Vol. 6015. LNCS. 2010, pp. 129–144 (cit. on p. 13).